Soluzioni — grep, sed, awk

File di riferimento: `sistema.log` | `utenti.txt` | `webserver.conf` | `vendite.csv`

grep

Es. 1 Mostrare tutte le righe di sistema.log che contengono la parola ERROR.

```
$ grep "ERROR" sistema.log
```

Es. 2 Mostrare le righe di sistema.log che contengono ERROR oppure WARNING.

```
$ grep -E "ERROR|WARNING" sistema.log
```

`-E` *abilita le espressioni regolari estese;* `|` *è l'operatore OR.*

Es. 3 Contare quante volte appare "Failed password" nel file di log.

```
$ grep -c "Failed password" sistema.log
```

`-c` *stampa il conteggio delle righe corrispondenti, non le righe stesse.*

Es. 4 In utenti.txt mostrare solo gli utenti la cui shell è /bin/bash.

```
$ grep "/bin/bash$" utenti.txt
```

Il `$` *ancora il pattern alla fine della riga: esclude eventuali righe che contengono* `/bin/bash` *nel mezzo del testo.*

Es. 5 In webserver.conf mostrare tutte le righe non vuote e non commentate.

```
$ grep -v "^#" webserver.conf | grep -v "^$"
```

Prima pipeline: esclude le righe che iniziano con `#`*. Seconda: esclude le righe vuote (*`^$` *= inizio riga immediatamente seguita da fine riga).*

sed

Es. 6 In webserver.conf, sostituire WARNING con ERROR solo a video.

```
$ sed 's/WARNING/ERROR/' webserver.conf
```

Senza -i l'output va sullo stdout; il file originale non viene modificato.

Es. 7 In webserver.conf, eliminare dall'output le righe di commento.

```
$ sed '/^#/d' webserver.conf
```

Il comando `d` *(delete) elimina le righe che corrispondono al pattern. Anche qui senza* `-i`*: nessuna modifica al file.*

Es. 8 In sistema.log, sostituire server01 con webserver.lan e salvare in un nuovo file.

```
$ sed 's/server01/webserver.lan/g' sistema.log > sistema_mod.log
```

Il flag `g` *sostituisce tutte le occorrenze per riga. Il reindirizzamento* `>` *salva l'output in* `sistema_mod.log` *senza toccare l'originale.*

awk

Es. 9 Da utenti.txt, estrarre nome utente (campo 1) e home directory (campo 6).

```
$ awk -F: '{print $1, $6}' utenti.txt
```

`-F:` *imposta i due punti come separatore di campo.*

Es. 10 Da vendite.csv, mostrare le righe della categoria "obiettivi" con prodotto e prezzo unitario.

```
$ awk -F, '$3 == "obiettivi" {print $2, $5}' vendite.csv
```

`$3` *è la categoria,* `$2` *il prodotto,* `$5` *il prezzo unitario.*

Es. 11 Da vendite.csv, calcolare il totale delle vendite per ogni venditore.

```
$ awk -F, 'NR>1 {totale[$7] += $6} END {for (v in totale) print v, totale[v]}'
vendite.csv
```

`NR>1` *salta la riga di intestazione.* `$7` *è il venditore,* `$6` *il totale della vendita. L'array associativo* `totale[]` *accumula la somma per ciascun venditore; il blocco* `END` *stampa i risultati a fine elaborazione.*

Es. 12 (avanzato) Da sistema.log, estrarre tutti gli IP e contare quante volte appare ciascuno.

```
$ grep -oE "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" sistema.log | sort | uniq -c | sort -rn
```

`-o` *stampa solo la parte corrispondente (non l'intera riga).* `-E` *abilita le regex estese per il pattern dell'IP.* `sort | uniq -c` *conta le occorrenze;* `sort -rn` *ordina per frequenza decrescente.*

Soluzione alternativa con solo awk:

```
$ awk 'match($0,/[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/) {ip=substr($0,RSTART,RLENGTH);
c[ip]++} END {for (ip in c) print c[ip], ip}' sistema.log | sort -rn
```